Umelé neurónové siete – MLP klasifikácia a aproximácia

Obsah

Úloha 5: Viacvrstvová perceptrónová sieť **2**
Cieľ úlohy . . . 2
Úloha 5a: Klasifikácia do skupín pomocou MLP siete . . . 2
Úloha 5b: Aproximácia nelineárnej funkcie pomocou MLP siete . . . 3

Odovzdanie a hodnotenie **4**
Dokumentácia výsledkov (PDF) . . . 4
Obhajoba riešenia . . . 4

Odporúčané nástroje a technická realizácia **5**
Matlab . . . 5
Python . . . 5

Teoretické poznámky **6**
Trénovacie, validačné a testovacie dáta . . . 6
Pretrénovanie a schopnosť generalizácie . . . 6
Počet neurónov v skrytej vrstve a schopnosť generalizácie . . . 7
Ako čítať grafy učenia . . . 8
Metriky chyby pri aproximácii . . . 8
Metriky pri klasifikácii a segmentácii . . . 8
Praktická poznámka k experimentovaniu . . . 8

Úloha 5: Viacvrstvová perceptrónová sieť (MLP)

Cieľ úlohy

Precvičte si dve demonštračné úlohy s MLP sieťou: (a) **klasifikáciu** bodov do tried a (b) **aproximáciu** nelineárnej funkcie. Programy a dáta k úlohám sú dostupné v AIS a v repozitároch predmetu (github). Ako doplnkový príklad/ukážka na klasifikáciu môžete použiť aj dataset **IRIS**, ktorý je dostupný v AIS a v repozitároch predmetu.

Úloha 5a: Klasifikácia do skupín pomocou MLP siete

Cieľ úlohy

Navrhnite a natrénujte MLP sieť, ktorá klasifikuje body do **5 tried** na základe vstupov x, y, z. Cieľom je nájsť čo najjednoduchšiu sieť (najmenší počet neurónov v skrytej vrstve) pri zachovaní čo najlepšej klasifikácie.

Postup riešenia

1. Použite dodaný Matlab program `klasifikuj_body_final.m` a dáta `databody.mat`. Pri riešení v Pythone použite dátový ekvivalent `databody.csv`.
2. Vytvorte MLP klasifikátor pomocou `patternet`. Dáta rozdeľte náhodným výberom, pričom trénovacia množina môže tvoriť maximálne 80% všetkých vzoriek. Zvyšok použite na testovanie a validáciu, prípadne len testovanie.
3. Nastavte a odlaďte počet neurónov v skrytej vrstve a ukončovacie podmienky trénovania tak, aby sieť správne klasifikovala čo najviac bodov. Akceptované sú najviac 2 zle klasifikované body.
4. Vykreslite priebeh chyby siete počas učenia (**loss vs. epoch**) a zobrazte kontingenčnú maticu cez `plotconfusion`.
5. Otestujte 5 definovaných bodov uvedených v programe. Pre každý bod uveďte výstup siete a výslednú priradenú triedu. Príslušnosť do skupín znázornite aj graficky.

Úloha 5b: Aproximácia nelineárnej funkcie pomocou MLP siete

Cieľ úlohy

Navrhnite a natrénujte MLP sieť, ktorá aproximuje nelineárnu funkciu $y = f(x)$ na základe meraných dát. Cieľom je dosiahnuť malú odchýlku na testovacích dátach a zároveň použiť čo najjednoduchšiu sieť.

Postup riešenia

1. Použite dodaný Matlab program `aproximuj_funkciu_final.m` a dáta `datafun.mat`. Pri riešení v Pythone použite dátové ekvivalenty `datafun.csv` a `datafunindx.csv`.
2. Rozdelenie na trénovacie a testovacie dáta realizujte podľa indexov `indx_train` a `indx_test` zo súboru `datafun.mat`. Pri riešení v Pythone použite rozdelenie zo súboru `datafunindx.csv`, kde **1** označuje trénovacie indexy a **2** testovacie indexy.
3. Vytvorte MLP sieť na aproximáciu pomocou `fitnet`. Nájdite optimálnu štruktúru siete (najmenší počet neurónov v skrytej vrstve) a nastavenie trénovania tak, aby ste dosiahli minimálnu odchýlku na testovacích dátach (pod 10^{-4}) a aby sa výstup siete tvarovo zhodoval s nameranými dátami.
4. Zdokumentujte proces trénovania: vykreslite priebeh chyby na trénovacích aj testovacích dátach (**loss vs. epoch**).
5. Ukážte aj prípady, kde sa sieť dobre nenaučila, napríklad pri príliš malom a pri príliš veľkom počte neurónov v skrytej vrstve. Pozorovania stručne zdôvodnite.
6. Vyčíslite SSE, MSE a MAE na trénovacích aj testovacích dátach. Porovnajte merané dáta s výstupom siete v jednom grafe a v grafe rozlíšte trénovacie dáta, testovacie dáta a výstup neurónovej siete.

Poznámka: Pod grafom **loss vs. epoch** sa myslí spoločný graf, v ktorom je zobrazený priebeh chyby počas učenia po jednotlivých epochách. Ak sa používajú validačné dáta, do jedného grafu vykreslite **trénovaciu aj validačnú chybu**. Testovacie dáta sa na priebežné učenie nepoužívajú, preto sa pri nich uvádza už len **výsledná chyba alebo metrika** dosiahnutá po natrénovaní siete. Keďže samotná hodnota **Loss / Stratová funkcia** nemusí byť pre človeka vždy ľahko interpretovateľná, odporúča sa podľa typu úlohy doplniť aj zrozumiteľnejšiu metriku, napríklad **Accuracy (acc) / Úspešnosť klasifikácie** pri klasifikácii.

Odovzdanie a hodnotenie

Toto zadanie slúži ako úvodné cvičenie k viacvrstvovým perceptrónovým sieťam a ich trénovaniu. **Zadanie nie je bodovo hodnotené** a neodovzdáva sa štandardným spôsobom ako ostatné zadania. Jeho cieľom je oboznámiť sa s pripravenými príkladmi, porozumieť základnej logike kódu a vytvoriť si podklad pre ďalšie úlohy v tomto tematickom bloku.

Odporúča sa úlohy samostatne spustiť, vyskúšať rôzne nastavenia a priebežne si ukladať výsledky. Na nasledujúcom cvičení je potom možné konzultovať prípadné nejasnosti, aby ďalšie zadania bolo možné riešiť už samostatnejšie a efektívnejšie.

Dokumentácia výsledkov (PDF)

Odporúča sa pripraviť si stručnú správu (napr. MS Word / PDF), ktorá bude obsahovať základné výstupy z oboch úloh a krátke poznámky k pozorovanému správaniu siete pri trénovaní. Dokument slúži predovšetkým ako vlastný podklad na pochopenie práce s neurónovou sieťou a na prípravu na ďalšie zadania.

Pri neurónových sieťach sa typicky uvádza priebeh učenia ako **loss vs. epoch** a podľa typu úlohy aj vhodné metriky. Treba počítať s tým, že medzi výsledkami v **Matlabe** a **Pythone** môže byť určitý rozdiel, pretože obe prostredia používajú odlišné implementačné detaily, prednastavenia a niektoré špecifické funkcie. Je preto prirodzené, že pri rovnakom zadaní nemusia viesť k úplne rovnakým priebehom ani číselným výsledkom.

Obhajoba riešenia

Formálna obhajoba tohto zadania sa **nevyžaduje**. Napriek tomu sa odporúča mať pripravené základné výstupy z oboch úloh a rozumieť tomu, čo jednotlivé časti kódu robia a aký význam majú hlavné výsledky, najmä grafy učenia, confusion matrix pri klasifikácii a chyby pri aproximácii.

Ak niektoré časti zadania alebo výsledkov nie sú jasné, odporúča sa pripraviť si konkrétne otázky a prekonzultovať ich na nasledujúcom cvičení.

Odporúčané nástroje a technická realizácia

Zadanie je možné vypracovať v prostredí **Matlab** alebo v jazyku **Python**. Ako alternatívu bez inštalácie je možné použiť **MATLAB Online** (`https://matlab.mathworks.com/`). Pre Python odporúčame **Google Colab** (`https://colab.research.google.com/`).

Matlab

Pri Matlabe použite vhodné nástroje pre neurónové siete podľa pokynov na cvičení. Do správy zahrňte priebeh učenia (loss vs. epoch) a výsledné metriky podľa zadania.

Pre Úlohu 5a použite `klasifikuj_body_final.m` a `databody.mat` (sieť cez `patternet`). Pre Úlohu 5b použite `aproximuj_funkciu_final.m` a `datafun.mat` (sieť cez `fitnet`).

Python

Pri Pythone odporúčame pracovať v prostredí Google Colab. Pre úlohy v tomto bloku použite oficiálne repozitáre:

- **Dodatočné kódy a zadania:** `https://github.com/STU-FEI-OUI/UMINT-UNS`
- **Dáta pre Python:** `https://github.com/STU-FEI-OUI/UMINT-UNS_data`

Pre vizualizáciu uveďte aspoň priebeh učenia ako **loss vs. epoch** a podľa zadania doplňte validáciu a metriky.

Pre Python sú dáta dostupné aj v CSV v repozitári **UMINT-UNS_data**. Úloha 5a používa `Body.csv`, kde stĺpce x, y, z sú vstupy a 4. stĺpec je `label`. Úloha 5b používa `datafun.csv` (stĺpce x, y) a `datafunindx.csv`, kde 2. stĺpec určuje rozdelenie (1 trénovacie dáta, 2 testovacie dáta).

Teoretické poznámky

Táto úloha slúži najmä ako **úvod do práce s viacvrstvovou perceptrónovou sieťou (MLP)** a na oboznámenie sa so základným procesom trénovania neurónových sietí. Cieľom preto nie je len „získať správny výsledok“, ale hlavne pochopiť, ako sa mení správanie siete pri zmene jej štruktúry a nastavení trénovania.

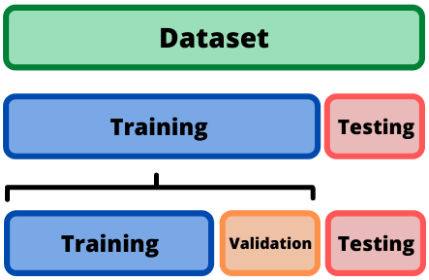
Pri klasifikácii sa sieť učí priraďovať vstupy do tried. Pri aproximácii sa sieť učí napodobniť priebeh neznámej funkcie na základe ukážkových dát. V oboch prípadoch je dôležité sledovať nielen finálny výsledok, ale aj **proces učenia**.

Trénovacie, validačné a testovacie dáta

Pri trénovaní neurónových sietí sa dáta zvyčajne delia na **trénovacie**, **validačné** a **testovacie**. Trénovacie dáta sa používajú priamo na úpravu váh siete. Validačné dáta sa používajú na priebežnú kontrolu, či sa sieť nezačína príliš prispôsobovať konkrétnym trénovacím vzorkám. Testovacie dáta slúžia až na finálne overenie, ako sa sieť správa na dátach, ktoré počas učenia „nevidela“.

Validačná množina sa pritom **spravidla vytvára z dát určených na trénovanie**, teda nejde o dáta, ktoré majú byť ponechané na finálne testovanie. Odporúčaný postup je mať **oddelené** trénovacie, validačné aj testovacie dáta, aby sa priebežné ladenie modelu neopieralo o tie isté dáta, na ktorých sa má na konci nezávisle overiť výsledok. Ak je dát málo, v jednoduchších ukážkach sa niekedy používa len rozdelenie na trénovacie a testovacie dáta a testovacia množina sa sleduje aj počas učenia, no ide skôr o zjednodušený postup a nie o odporúčaný spôsob vyhodnotenia.

Dobrý výsledok na trénovacích dátach ešte automaticky neznamená, že sa sieť naučila problém riešiť všeobecne. Práve preto je dôležité sledovať rozdiel medzi správaním siete na trénovacích a testovacích dátach. Pri porovnaní trénovacích a testovacích dát sledujte aj to, či sa neobjavujú známky pretrénovania, teda situácia, keď sa sieť ďalej zlepšuje na trénovacích dátach, ale už nie na dátach validačných alebo testovacích.



Obr. 1: Ilustrácia typického rozdelenia dát na trénovaciu, validačnú a testovaciu časť.

Pretrénovanie a schopnosť generalizácie

Pri neurónových sieťach nestačí sledovať len to, ako dobre sa sieť naučila **trénovacie dáta**. Dôležité je aj to, ako sa správa na **nových dátach**, ktoré počas učenia nevidela. Práve táto schopnosť sa označuje ako **generalizácia**.

K **pretrénovaniu** (overfitting) dochádza vtedy, keď sa sieť príliš prispôsobí trénovacím dátam, vrátane ich šumu alebo náhodných odchýlok. Navonok to môže vyzerať tak, že chyba na trénovacích dátach ďalej klesá, ale na validačných alebo testovacích dátach sa výsledok už nezlepšuje, prípadne sa začne zhoršovať. Takáto sieť síce dobre „pozná“ trénovacie dáta, ale horšie sa správa na nových vstupoch.

V praxi sa preto hľadá kompromis medzi jednoduchosťou siete, dĺžkou učenia a dosiahnutou presnosťou. Príliš jednoduchá sieť môže mať problém zachytiť vzťahy v dátach, príliš zložitá sa

môže naučiť trénovacie dáta príliš presne, ale bez dostatočnej schopnosti generalizácie.

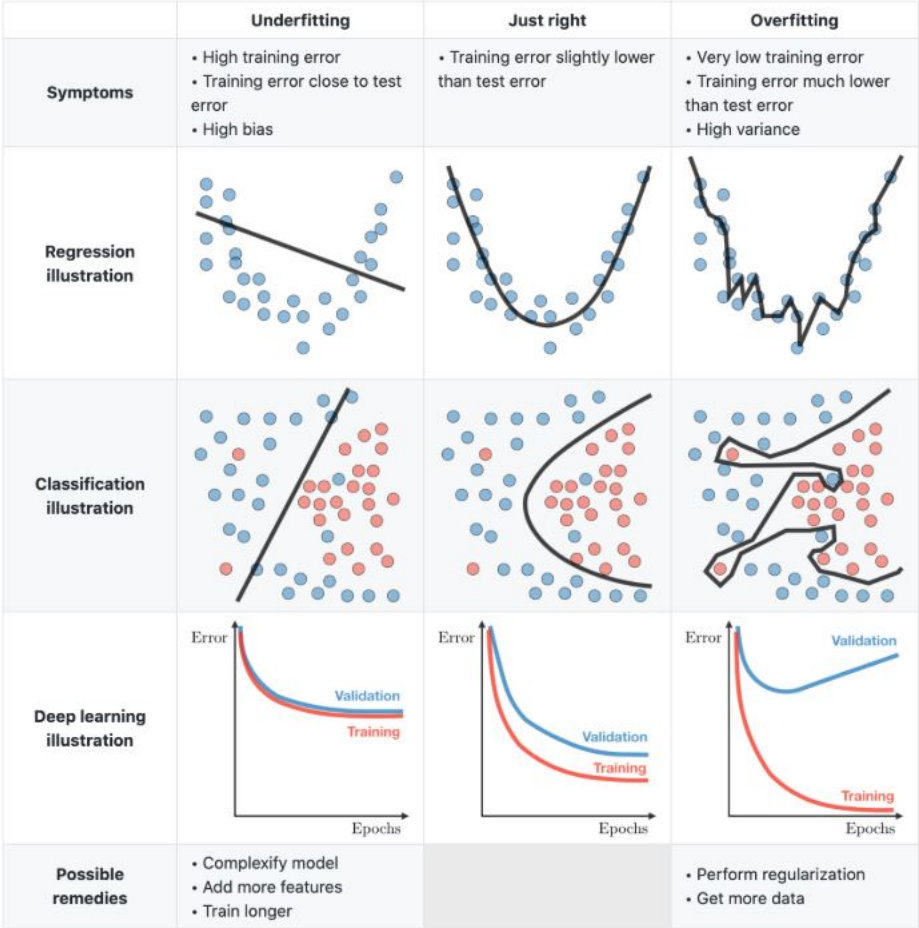
K **pretrénovaniu** (overfitting) dochádza vtedy, keď sa sieť príliš prispôsobí trénovacím dátam, vrátane ich šumu alebo náhodných odchýlok. Navonok to môže vyzerať tak, že chyba na trénovacích dátach ďalej klesá, ale na validačných alebo testovacích dátach sa výsledok už nezlepšuje, prípadne sa začne zhoršovať. Takáto sieť síce dobre „pozná“ trénovacie dáta, ale horšie sa správa na nových vstupoch.

V praxi sa preto hľadá kompromis medzi jednoduchosťou siete, dĺžkou učenia a dosiahnutou presnosťou. Príliš jednoduchá sieť môže mať problém zachytiť vzťahy v dátach, príliš zložitá sa môže naučiť aj to, čo generalizovať nemá.

Počet neurónov v skrytej vrstve a schopnosť generalizácie

Počet neurónov v skrytej vrstve výrazne ovplyvňuje správanie siete. Ak je neurónov príliš málo, sieť nemusí mať dostatočnú kapacitu na zachytenie vzťahov v dátach. Ak je neurónov príliš veľa, sieť sa môže naučiť trénovacie dáta príliš presne, ale na nových dátach sa bude správať horšie.

V tejto úlohe preto nehľadáte len „najlepší výsledok“, ale aj **rozumný kompromis medzi jednoduchosťou siete a kvalitou výsledku**. Pri diskusii má zmysel sledovať, ako sa mení priebeh loss a výsledné chyby pri malej, primeranej a zbytočne veľkej sieti.



Obr. 2: Ilustrácia rozdielu medzi nedostatočným naučením, primeraným naučením a pretrénovaním modelu.

Ako čítať grafy učenia

Graf **loss vs. epoch** ukazuje, ako sa mení chyba siete počas trénovania. Vo všeobecnosti platí, že ak loss klesá, sieť sa učí. Nie je však dôležitá len samotná hodnota loss, ale aj tvar priebehu. Zaujímavé je napríklad, či chyba klesá plynulo, či sa skoro zastaví, alebo či sa správanie trénovacích a testovacích dát začne výrazne odlišovať.

Pri klasifikácii preto sledujte aj **confusion matrix**, pretože tá ukazuje, medzi ktorými triedami si sieť robí chyby. Pri aproximácii nestačí len číselná chyba. Dôležité je aj to, či sa tvar funkcie vypočítaný sieťou vizuálne zhoduje s pôvodnými dátami.

Metriky chyby pri aproximácii

Pri aproximačnej úlohe sa používajú metriky **SSE**, **MSE** a **MAE**. Tieto metriky popisujú odchýlku medzi výstupom siete a očakávanými hodnotami, ale každá trochu iným spôsobom.

SSE zdôrazňuje celkovú veľkosť chyby. MSE ju spriemeruje na jednu vzorku. MAE vyjadruje priemernú absolútnu odchýlku a býva ľahšie interpretovateľná. Pri porovnávaní rôznych behov alebo rôznych štruktúr siete preto má zmysel sledovať viac než jednu metriku.

Metriky pri klasifikácii a segmentácii

Pri klasifikačných úlohách sa počas učenia často optimalizuje **Loss / Stratová funkcia**, napríklad **cross-entropy**, pretože je vhodná na trénovanie siete a dobre reaguje na zmenu predikovaných pravdepodobností. Jej číselná hodnota však nemusí byť pre človeka vždy intuitívna, preto sa popri nej často zobrazuje aj **Accuracy (acc) / Úspešnosť klasifikácie** ako jednoduchší ukazovateľ toho, aký podiel vzoriek bol klasifikovaný správne.

Accuracy / Úspešnosť klasifikácie teda neslúži primárne ako samotný tréningový cieľ, ale ako doplnková metrika, ktorá umožňuje jednoduchšie pochopiť, či sa klasifikácia zlepšuje. Pri vyvážených triedach býva užitočná, no sama osebe nemusí stačiť na úplné posúdenie kvality modelu.

Pri zložitejších úlohách, napríklad pri **segmentácii**, sa používajú aj ďalšie metriky. Často sa sleduje **F1-score** (resp. Dice koeficient), ktorý lepšie vystihuje prekryv medzi očakávaným a predikovaným výsledkom. Používa sa aj **IoU** (Intersection over Union), ktorá vyjadruje mieru prekryvu segmentovaných oblastí. V niektorých prípadoch sa môže uvádzať aj **MSE**, najmä ak je výstup chápaný ako číselná aproximácia masky alebo pravdepodobnostnej mapy, no pri klasifikácii a segmentácii býva vhodnejšie doplniť metriky, ktoré lepšie zodpovedajú samotnému rozhodovaniu modelu.

Pri interpretácii výsledkov má preto zmysel rozlišovať medzi tým, čo sa počas učenia optimalizuje ako **loss**, a tým, čo sa následne zobrazuje ako zrozumiteľnejšia **metrika kvality výsledku**.

Praktická poznámka k experimentovaniu

Keďže ide o úvodnú a nehodnotenú úlohu, odporúča sa **experimentovať** a cielene skúšať (pocet epoch, learing rate a ine) aj menej vhodné nastavenia. Má zmysel porovnať napríklad menší a väčší počet neurónov, odlišné ukončenie trénovania alebo iné rozdelenie dát. Práve tieto porovnania zvyčajne najlepšie ukážu, ako neurónová sieť funguje a čo vplýva na jej výsledok.